

Vous devez penser que ce exercice est facile et qu'il suffit juste d'utiliser le printf pour afficher des caractères mais le problème est qu'on ne dispose pas du printf

Pourquoi on ne peut pas utiliser printf (ou n'importe quelle fonction de la libc) ?

Dans un kernel que nous écrivons depuis zéro :

- Il n'y a **pas d'OS hôte** (pas de Linux, pas de Windows, pas de libc préchargée).
- La libc standard (glibc, musl, newlib...) suppose qu'il y a déjà un système d'exploitation qui fournit :
  - des syscalls (write, ioctl...)
  - un driver console
  - une gestion de mémoire
  - des interruptions gérées
- Or, au stade où on est (kernel brut multiboot), **rien de tout ça n'existe encore**.

Donc printf → crash ou lien impossible (undefined reference to printf).

Il faut **donc tout réécrire soi-même**

```

#define FB_COLS 80
#define FB_ROWS 25
#define VIDEO_ADDRESS 0xB8000

// Variable globale pour le pilote : garde la trace de la position
unsigned int cursor_pos = 0;

void fb_write_cell(unsigned int i, char c, unsigned char fg, unsigned char bg) {
    char *fb = (char *) VIDEO_ADDRESS;
    fb[i * 2] = c;
    fb[i * 2 + 1] = ((bg & 0x0F) << 4) | (fg & 0x0F);
}

// Fonction du pilote : on va les mettre dans un fichier separé apres
void fb_put_char(char c) {
    if (c == '\n') { // Gestion rudimentaire du retour à la ligne
        cursor_pos = ((cursor_pos / FB_COLS) + 1) * FB_COLS;
    } else {
        fb_write_cell(cursor_pos, c, 2, 0); // Vert sur Noir
        cursor_pos++;
    }

    // Sécurité : si on dépasse l'écran, on revient en haut
    if (cursor_pos >= FB_COLS * FB_ROWS) {
        cursor_pos = 0;
    }
}

void fb_print(const char *str) {
    for (int i = 0; str[i] != '\0'; i++) {
        fb_put_char(str[i]);
    }
}

void kmain() {
    // 1. Je vais faire une fonction qui va le faire l'effacement apres pour que ce soit elegant
    for(int i = 0; i < FB_COLS * FB_ROWS; i++) {
        fb_write_cell(i, ' ', 0, 0);
    }

    // 2. Utilisation du pilote
    fb_print("Pilote Framebuffer: Charge\n");
    fb_print("Ligne 2: OK\n");
    fb_print("Ring 0: Acces total a 0xB8000");
}

```

NB : Pour des soucis de modularité on va diviser le code après pour avoir le pilotes à part et le main à part

```

one@DESKTOP-5RV2RM1:/mnt/c/Users/MOH/Documents/Lab1_CYBER/ProjetOS2$ qemu-system-i386 -cdrom os.iso -nographic -serial null -monitor stdio
QEMU 8.2.2 monitor - type 'help' for more information
(qemu) info registers

CPU#0
EAX=00000000 EBX=0000d900 ECX=00008a0e EDX=00000000
ESI=00000fe18 EDI=00000000 EBP=00001ff0 ESP=00001fe6
EIP=00008a2c EFL=00000246 [---Z-P-] CPL=0 II=0 A20=1 SMM=0 HLT=0
ES =0040 00000400 0000ffff 00009300
CS =f000 000f0000 0000ffff 00009e00
SS =0000 00000000 0000ffff 00009300
DS =0000 00000000 0000ffff 00009300
FS =0000 00000000 0000ffff 00009300
GS =0000 00000000 0000ffff 00009300
LDT=0000 00000000 0000ffff 00008200
TR =0000 00000000 0000ffff 00008b00
GDT= 00008280 00000027
IDT= 00000000 000003ff
CR0=00000010 CR2=00000000 CR3=00000000 CR4=00000000
DR0=00000000 DR1=00000000 DR2=00000000 DR3=00000000
DR6=ffff0fff DR7=00000400
EFER=0000000000000000

```

```

FPR4=0000000000000000 0000 FPR5=0000000000000000 0000
FPR6=0000000000000000 0000 FPR7=0000000000000000 0000
XMM00=0000000000000000 0000000000000000 XMM01=0000000000000000 0000000000000000
XMM02=0000000000000000 0000000000000000 XMM03=0000000000000000 0000000000000000
XMM04=0000000000000000 0000000000000000 XMM05=0000000000000000 0000000000000000
XMM06=0000000000000000 0000000000000000 XMM07=0000000000000000 0000000000000000
(qemu) info registers

CPU#0
EAX=00000100 EBX=0000df7e ECX=00000000 EDX=07f977e0
ESI=0007fedc EDI=07f97840 EBP=00001ff0 ESP=00001ff0
EIP=00009102 EFL=00000202 [-----] CPL=0 II=0 A20=1 SMM=0 HLT=0
ES =7840 00078400 0000ffff 00009300
CS =0000 00000000 0000ffff 00009e00
SS =0000 00000000 0000ffff 00009300
DS =07f9 00007f90 0000ffff 00009300
FS =0000 00000000 0000ffff 00009300
GS =0000 00000000 0000ffff 00009300
LDT=0000 00000000 0000ffff 00008200
TR =0000 00000000 0000ffff 00008b00
GDT= 00008280 00000027
IDT= 00000000 000003ff
CR0=00000010 CR2=00000000 CR3=00000000 CR4=00000000
DR0=00000000 DR1=00000000 DR2=00000000 DR3=00000000
DR6=ffff0fff DR7=00000400
EFER=0000000000000000
FCW=037f FSW=0000 [ST=0] FTW=00 MXCSR=00001f80

```

Cela prouve qu'une exécution a bien été faite car le registre EIP (*Extended Instruction Pointer*) pointe sur l'adresse de la prochaine instruction donc si on va passer à une autre instruction bien sûr que le EIP va changer.

Pour voir ce qui se passe vraiment on va juste enlever l'option -nographic sur la commande qemu

Machine View

```
Pilote Framebuffer: Charge  
Ligne 2: OK  
Ring 0: Acces total a 0xB8000
```

```
void fb_move_cursor(unsigned short pos) {  
    outb(FB_COMMAND_PORT, FB_HIGH_BYTE_COMMAND);  
    outb(FB_DATA_PORT, ((pos >> 8) & 0x00FF));  
    outb(FB_COMMAND_PORT, FB_LOW_BYTE_COMMAND);  
    outb(FB_DATA_PORT, (pos & 0x00FF));  
}
```

Ceci est la fonction qui se charge de déplacer le curseur conformément à l'instruction du kmain.c qui fichier de combien d'octet on veut déplacer.

```
one@DESKTOP-5RV2RM1:/mnt/c/Users/MOH/Documents/Lab1_CYBER/ProjetOS2$ tree
```

```
├── framebuffer.c
├── io.h
├── io.o
├── io.s
├── iso
│   └── boot
│       ├── grub
│       │   ├── menu.lst
│       │   └── stage2_eltorito
│       └── kernel.elf
├── kmain.c
├── kmain.c.backup
├── kmain.o
├── link.ld
├── loader.o
├── loader.s
└── os.iso
```

4 directories, 14 files

QEMU - Press Ctrl+Alt+G to release grab

×

Machine View

ProjetOS

Module Framebuffer: Initialise

Statut: Ring 0 Kernel Ready.

ca commence a ressembler a quelque chose

QEMU - Press Ctrl+Alt+G to release grab

×

Machine View

ProjetOS

Module Framebuffer: Initialise

Statut: Ring 0 Kernel Ready.

ca commence a ressembler a quelque chose

QEMU - Press Ctrl+Alt+G to release grab

×

Machine View

ProjetOS

Module Framebuffer: Initialise

Statut: Ring 0 Kernel Ready.

ca commence a ressembler a quelque chose

Juste regardez sous le "c" au début de la dernier ligne on voir bien que ça clignote.